CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 09

Deadlock: Part 1



Deadlock: Part 1

Topics to be Covered

1. Definition
2. Necessary Conditions
3. Resource Allocation Graph
4. Process Wait-for Graph
5. Methods of Deadlock Handling
6. Banker's Algorithm

Definition of Deadlock

Deadlock:

★ A **deadlock** is a condition in which a **set of processes** is waiting **forever** for resources held by each other.

★ None of the processes can proceed with their execution because **each is waiting for a resource occupied by another process**.

Necessary Conditions for Deadlock

A **deadlock** can occur only if the following **four conditions** hold **simultaneously**:

1. Mutual Exclusion

★ At least one resource must be held in a **non-shareable (exclusive)** mode.

★ Only one process can use the resource at a time.

2. Hold and Wait

★ A process is **holding** at least one resource and **waiting** to acquire additional resources that are currently held by other processes.

3. No Preemption

★ Resources **cannot be forcibly taken** from a process; a process must **release the resource voluntarily** after completing its task.

4. Circular Wait (Cycle Wait)

★ A **set of processes** $\{P_0, P_1, P_2, \ldots, P_{n-1}\}$ must exist such that:

- P_0 is waiting for a resource held by P_1,
- P_1 is waiting for a resource held by P_2,
- and so on,
- until P_{n-1} waits for a resource held by P_0.

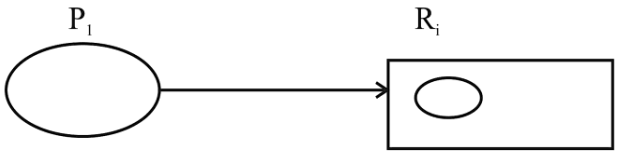
Note:

- These are **necessary but not sufficient** conditions for deadlock — even if all four hold, deadlock may or may not occur.

Resource Allocation Graph (RAG)

A **Resource Allocation Graph** is used to represent resource allocation and process requests in a system.

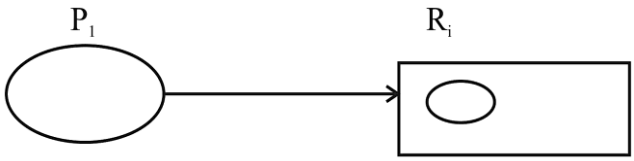
Components of RAG



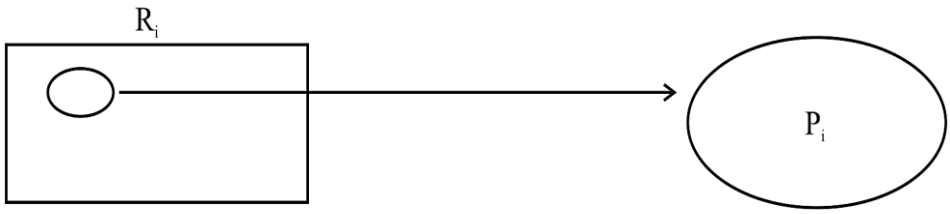
- **Processes (P_1, P_2, …)** → represented by circles.
- **Resources (R_1, R_2, …)** → represented by rectangles.

Types of Edges

1. **Request Edge (Process → Resource):**
   - Indicates that a process has requested a resource.
   - Example: $P_1 \rightarrow R_1$ means process P_1 is requesting resource R_1.

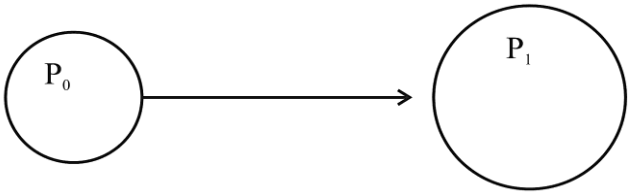


2. **Assignment Edge (Resource → Process):**
   - Indicates that a resource is currently **allocated** to a process.
   - Example: $R_1 \rightarrow P_1$ means resource R_1 is assigned to process P_1.



Process Wait-for Graph (PWFG)

- A **Process Wait-for Graph** is derived from a Resource Allocation Graph.
- It is used to **simplify deadlock detection** when each resource type has only **one instance**.
- In PWFG:
  - **Nodes** represent processes.
  - **Edges** represent waiting relationships.



Definition:

★ An edge from $P_0 \rightarrow P_1$ means P_0 **is waiting** for a resource that is currently held by P_1.

★ If the PWFG contains a **cycle**, a **deadlock exists**.

Methods of Deadlock Handling

There are **three main methods** for handling deadlocks:

1. **Deadlock Prevention**
   - ★ The system is designed in such a way that **at least one of the four necessary conditions** for deadlock **can never hold**.
   - ★ If even one condition is prevented, a deadlock **cannot occur**.

   Example:
   - ★ Avoid **Hold and Wait** by requiring processes to **request all resources at once**.
   - ★ Avoid **Circular Wait** by **ordering resource requests**.

2. Deadlock Avoidance

★ The system **monitors every resource request** dynamically and **decides** whether to grant it based on system safety.

★ The system is in a **safe state** if all pending processes can complete in some order **without deadlock**.

Key Terms:

★ **Safe State:** All processes can complete execution in a safe sequence.

★ **Unsafe State:** Deadlock may occur, but not necessarily.

★ **Safe Sequence:** The order in which processes can safely execute while meeting all resource needs.

3. Deadlock Detection and Recovery

Used when prevention or avoidance is **too costly** or **complex**.

Steps:

a. **Allow deadlock to occur.**

b. **Detect** the deadlock when it occurs.

c. **Recover** from the deadlock situation.

Allow Deadlock to Occur

★ The system allows deadlock and **restarts** when it stops functioning.

★ Suitable for **non-critical applications**.

★ This is the **least costly** method, but not suitable for critical or real-time systems.

Banker's Algorithm

★ Banker's Algorithm is a **deadlock avoidance algorithm**.

★ It keeps the system always in a **safe state**.

★ Named after the banking system where a banker never allocates cash in a way that could cause insolvency.

Working Steps

(a) When a process enters the system, it must **declare its maximum possible resource requirement** for its entire lifetime.

★ The system will admit a process **only if** its declared maximum requirement can be safely fulfilled.

★ The OS checks if, after allocation, the system remains in a **safe state**.

★ If yes → allocation is granted.

★ If no → the process must **wait**.

(b) When a process requests resources, the system checks:

★ Whether the request is **within its declared maximum**.

★ Whether granting it will keep the system in a **safe state**.

If both are true, the allocation is made; otherwise, the process waits.

Example of Banker's Algorithm

Question:

Find the **Safe Sequence** for the given data.

Process	Allocation	Maximum
P_0	5	10
P_1	4	6
P_2	1	2

Total Resources = 13

Solution:

Find Free Resources:

Free = Total - Allocated

Free = 13 - (5 + 4 + 1) = 3

Calculate Need (Max – Allocation):

Process	Allocation	Maximum	Need
P_0	5	10	5
P_1	4	6	2
P_2	1	2	1

Step-by-Step Safe Sequence Calculation

- **Initial Free = 3**
  1. P_2 can be executed (Need = $1 \leq 3$)
     → After execution: Free = 3 + 1 = 4
  2. P_1 can now execute (Need = $2 \leq 4$)
     → After execution: Free = 4 + 4 = 8
  3. P_0 can now execute (Need = $5 \leq 8$)
     → After execution: Free = 8 + 5 = 13

  Safe Sequence: <P_2, P_1, P_0>

Summary of Deadlock

Concept	Description / Key Points
Deadlock Definition	Processes are waiting forever for resources held by others.
Necessary Conditions	Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.
Resource Allocation Graph	Visual representation of resource allocation and requests.
Process Wait-for Graph	Shows which processes are waiting for others; cycle = deadlock.
Methods of Handling	Prevention, Avoidance, Detection & Recovery.
Safe Sequence	An order in which all processes can execute safely.
Banker's Algorithm	Deadlock avoidance method maintaining safe state; ensures safe allocation.

डेडलॉक: भाग 1

शामिल किए जाने वाले विषय

1. परिभाषा
2. आवश्यक शर्तें
3. रिसोर्स एलोकेशन ग्राफ़
4. प्रोसेस वेट-फ़ॉर ग्राफ़
5. डेडलॉक हैंडलिंग के तरीके
6. बैंकर एल्गोरिदम

डेडलॉक की परिभाषा

डेडलॉक:

★ **डेडलॉक** एक ऐसी स्थिति है जिसमें **प्रोसेस का एक सेट** एक-दूसरे द्वारा रखे गए रिसोर्स के लिए **हमेशा** इंतजार करता रहता है।

★ कोई भी प्रोसेस अपना एग्जीक्यूशन आगे नहीं बढ़ा सकता क्योंकि **हर कोई दूसरे प्रोसेस द्वारा इस्तेमाल किए जा रहे रिसोर्स का इंतजार कर रहा है।**

डेडलॉक के लिए आवश्यक शर्तें

डेडलॉक तभी हो सकता है जब ये चार शर्तें एक साथ पूरी हों:

1. **म्यूचुअल एक्सक्लूज़न**
   ★ कम से कम एक रिसोर्स **नॉन-शेयरबल (एक्सक्लूसिव)** मोड में होना चाहिए।
   ★ एक समय में केवल एक प्रोसेस ही रिसोर्स का इस्तेमाल कर सकता है।

2. **होल्ड और वेट**
   ★ एक प्रोसेस कम से कम एक रिसोर्स को **होल्ड कर रहा है** और दूसरे रिसोर्स पाने का **इंतज़ार कर रहा है** जो अभी दूसरे प्रोसेस के पास हैं।

3. **नो प्रीएम्पशन**
   ★ रिसोर्स किसी प्रोसेस से **ज़बरदस्ती नहीं लिए जा सकते**; एक प्रोसेस को अपना काम पूरा करने के बाद **रिसोर्स को अपनी मर्ज़ी से छोड़ना होगा।**

4. **सर्कुलर वेट (साइकिल वेट)**
   ★ **प्रोसेस का एक सेट** $\{P_0, P_1, P_2, \ldots, P_{n-1}\}$ ऐसा होना चाहिए कि:
      ○ P_0, P_1 के पास मौजूद रिसोर्स का इंतज़ार कर रहा हो,
      ○ P_1, P_2 के पास मौजूद रिसोर्स का इंतज़ार कर रहा हो,
      ○ और इसी तरह,
      ○ जब तक P_{n-1} P_0 के पास मौजूद रिसोर्स का इंतज़ार न करे।

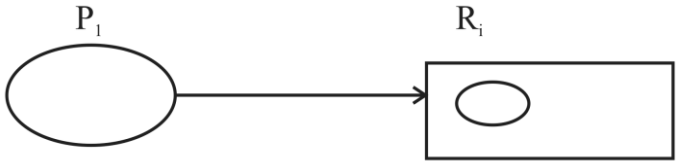
नोट:

- ये डेडलॉक के लिए **ज़रूरी शर्तें हैं, लेकिन काफ़ी नहीं** — भले ही ये चारों शर्तें पूरी हों, डेडलॉक हो भी सकता है और नहीं भी।

रिसोर्स एलोकेशन ग्राफ़ (RAG)

एक सिस्टम में रिसोर्स एलोकेशन और प्रोसेस रिक्वेस्ट को दिखाने के लिए **रिसोर्स एलोकेशन ग्राफ़** का इस्तेमाल किया जाता है।

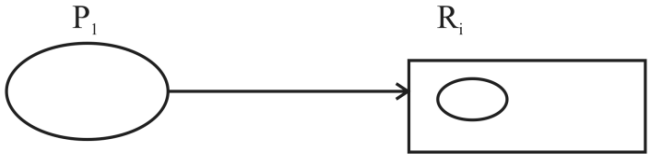
RAG के घटक



- **प्रोसेस** ($P_1, P_2, \ldots$) → गोलों से दिखाए जाते हैं।
- **रिसोर्स** ($R_1, R_2, \ldots$) → आयतों से दिखाए जाते हैं।

एज के प्रकार

1. **रिक्वेस्ट एज (प्रोसेस → रिसोर्स):**
   - यह बताता है कि किसी प्रोसेस ने किसी रिसोर्स के लिए रिक्वेस्ट की है।
   - उदाहरण: $P_1 \rightarrow R_1$ का मतलब है कि प्रोसेस P_1 रिसोर्स R_1 के लिए रिक्वेस्ट कर रहा है।



2. **असाइनमेंट एज (रिसोर्स → प्रोसेस):**
   - यह बताता है कि एक रिसोर्स अभी किसी प्रोसेस को दिया गया है।
   - **उदाहरण:** $R_1 \rightarrow P_1$ का मतलब है कि रिसोर्स R_1 प्रोसेस P_1 को असाइन किया गया है।



प्रोसेस वेट-फॉर ग्राफ़ (PWFG)

- एक **प्रोसेस वेट-फॉर ग्राफ़** रिसोर्स एलोकेशन ग्राफ़ से बनाया जाता है।
- इसका इस्तेमाल **डेडलॉक का पता लगाने को आसान बनाने के लिए** किया जाता है, जब हर रिसोर्स टाइप का सिर्फ़ **एक इंस्टेंस** होता है।
- PWFG में:
  - **नोड्स** प्रोसेस को दिखाते हैं।
  - **एज** वेटिंग रिलेशनशिप को दिखाते हैं।



परिभाषा:

★ $P_0 \rightarrow P_1$ का मतलब है कि **P_0 एक ऐसे रिसोर्स का इंतज़ार कर रहा** है जो अभी **P_1** के पास है।

★ अगर PWFG में कोई **साइकिल** है, तो **डेडलॉक मौजूद** है।

डेडलॉक हैंडलिंग के तरीके

डेडलॉक हैंडलिंग के **तीन मुख्य तरीके** हैं:

1. **डेडलॉक की रोकथाम (डेडलॉक प्रिवेंशन)**
   - ★ सिस्टम को इस तरह से डिज़ाइन किया गया है कि डेडलॉक के लिए **चार ज़रूरी शर्तों में से कम से कम एक शर्त कभी पूरी नहीं हो सकती।**
   - ★ अगर एक भी शर्त को रोक दिया जाए, तो डेडलॉक **नहीं हो सकता।**

उदाहरण:

- ★ प्रोसेस को **एक साथ सभी रिसोर्स का अनुरोध** करने के लिए कहकर **होल्ड एंड वेट** से बचें।
- ★ **रिसोर्स अनुरोधों को ऑर्डर** देकर **सर्कुलर वेट** से बचें।

2. **डेडलॉक से बचाव (डेडलॉक अवॉयडेंस)**
   - ★ सिस्टम **हर रिसोर्स अनुरोध को डायनामिक रूप से मॉनिटर करता है** और सिस्टम की सुरक्षा के आधार पर यह तय करता है कि उसे मंज़ूरी देनी है या नहीं।
   - ★ सिस्टम **सुरक्षित स्थिति** में होता है यदि सभी पेंडिंग प्रोसेस **बिना डेडलॉक के** किसी क्रम में पूरे हो सकते हैं।

   मुख्य शब्द:
   - ★ **सेफ स्टेट:** सभी प्रोसेस एक सेफ सीक्वेंस में एग्जीक्यूशन पूरा कर सकते हैं।
   - ★ **अनसेफ स्टेट:** डेडलॉक हो सकता है, लेकिन ज़रूरी नहीं।
   - ★ **सेफ सीक्वेंस:** वह क्रम जिसमें प्रोसेस सभी रिसोर्स की ज़रूरतों को पूरा करते हुए सुरक्षित रूप से एग्जीक्यूट हो सकते हैं।

3. **डेडलॉक का पता लगाना और उससे उबरना (डेडलॉक डिटेक्शन और रिकवरी)**

   इसका इस्तेमाल तब किया जाता है जब रोकथाम या **बचाव बहुत** महंगा या **जटिल** हो।

   चरण:
   a. **डेडलॉक होने दें।**
   b. जब डेडलॉक हो, तो उसे **डिटेक्ट** करें।
   c. डेडलॉक की स्थिति से **रिकवर** करें।

   डेडलॉक होने दें
   - ★ सिस्टम डेडलॉक होने देता है और जब काम करना बंद कर देता है तो **रीस्टार्ट** हो जाता है।
   - ★ **नॉन-क्रिटिकल एप्लीकेशन** के लिए उपयुक्त।
   - ★ यह सबसे **कम खर्चीला** तरीका है, लेकिन क्रिटिकल या रियल-टाइम सिस्टम के लिए उपयुक्त नहीं है।

बैंकर एल्गोरिदम

- ★ बैंकर एल्गोरिदम एक **डेडलॉक से बचने का एल्गोरिदम** है।
- ★ यह सिस्टम को हमेशा **सुरक्षित स्थिति** में रखता है।
- ★ इसका नाम बैंकिंग सिस्टम के नाम पर रखा गया है, जहाँ एक बैंकर कभी भी इस तरह से कैश एलोकेट नहीं करता जिससे दिवालियापन हो जाए।

कार्यविधि

(a) जब कोई प्रोसेस सिस्टम में आता है, तो उसे अपनी पूरी लाइफटाइम के लिए अपनी **ज़्यादा से ज़्यादा रिसोर्स की ज़रूरत बतानी होगी।**

- ★ सिस्टम किसी प्रोसेस को तभी एडमिट करेगा **जब** उसकी बताई गई ज़्यादा से ज़्यादा ज़रूरत को सुरक्षित रूप से पूरा किया जा सके।
- ★ OS चेक करता है कि एलोकेशन के बाद, सिस्टम **सेफ स्टेट** में रहता है या नहीं।
- ★ अगर हाँ → एलोकेशन मिल जाता है।
- ★ अगर नहीं → प्रोसेस को **इंतज़ार** करना होगा।

(b) जब कोई प्रोसेस रिसोर्स के लिए रिक्वेस्ट करता है, तो सिस्टम चेक करता है:

- ★ क्या रिक्वेस्ट उसकी बताई गई **ज़्यादा से ज़्यादा लिमिट के अंदर है।**
- ★ क्या इसे देने से सिस्टम **सेफ स्टेट** में रहेगा।

अगर दोनों सच हैं, तो एलोकेशन हो जाता है; नहीं तो, प्रोसेस इंतज़ार करता है।

बैंकर एल्गोरिदम का उदाहरण

प्रश्न:

दिए गए डेटा के लिए **सुरक्षित अनुक्रम** ज्ञात कीजिए।

प्रोसेस	**एलोकेशन**	**मैक्सिमम**
P_0	5	10
P_1	4	6
P_2	1	2

कुल रिसोर्स = 13

हल:

फ्री रिसोर्सेस ज्ञात करें:

फ्री = कुल - एलोकेटेड

फ्री = 13 – (5 + 4 + 1) = 3

नीड (मैक्सिमम – एलोकेशन) की गणना करें:

प्रोसेस	**एलोकेशन**	**मैक्सिमम**	**नीड**
P_0	5	10	5
P_1	4	6	2
P_2	1	2	1

स्टेप-बाय-स्टेप सेफ सीक्वेंस कैलकुलेशन

- **शुरुआती फ्री = 3**

1. P_2 को एग्जीक्यूट किया जा सकता है (नीड = $1 \leq 3$)
→ एग्जीक्यूशन के बाद: फ्री = 3 + 1 = 4

2. अब P_1 एग्जीक्यूट हो सकता है (नीड = $2 \leq 4$)
→ एग्जीक्यूशन के बाद: फ्री = 4 + 4 = 8

3. अब P_0 एग्जीक्यूट हो सकता है (नीड = $5 \leq 8$)
→ एग्जीक्यूशन के बाद: फ्री = 8 + 5 = 13

सेफ सीक्वेंस: <P_2, P_1, P_0>

डेडलॉक का सारांश

अवधारणा	विवरण / मुख्य बिंदु
डेडलॉक परिभाषा	प्रोसेस दूसरों द्वारा रखे गए रिसोर्स के लिए हमेशा इंतजार कर रहे हैं।
आवश्यक शर्तें	म्यूचुअल एक्सक्लूजन, होल्ड एंड वेट, नो प्रीएम्प्शन, सर्कुलर वेट।
रिसोर्स एलोकेशन ग्राफ	रिसोर्स एलोकेशन और रिक्वेस्ट का दृश्य निरूपण।
प्रोसेस वेट-फॉर ग्राफ़	दिखाता है कि कौन से प्रोसेस दूसरों का इंतज़ार कर रहे हैं; साइकिल = डेडलॉक।
हैंडलिंग के तरीके	रोकथाम, बचाव, पता लगाना और रिकवरी।
सेफ सीक्वेंस	एक ऐसा क्रम जिसमें सभी प्रोसेस सुरक्षित रूप से चल सकते हैं।
बैंकर एल्गोरिदम	सेफ स्टेट बनाए रखते हुए डेडलॉक से बचने का तरीका; सुरक्षित एलोकेशन सुनिश्चित करता है।



PW Web/App: https://smart.link/7wwosivoicgd4

Library- https://smart.link/sdfez8ejd80if